Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-211БВ-25

Студент: Щепинов Д.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 16.10.26

Москва, 2026

Постановка задачи

Вариант 4.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса пишет имя файла, которое будет передано при создании дочернего процесса. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс передает команды пользователя через pipe1, который связан с стандартным входным потоком дочернего процесса. Дочерний процесс принеобходимости передает данные в родительский процесс через pipe2. Результаты своей работы дочерний процесс пишет в созданный им файл. Допускается просто открыть файл и писать туда, не перенаправляя стандартный поток вывода.

Пользователь вводит команды вида: «число число число». Далее эти числа передаются от родительского процесса в дочерний. Дочерний процесс производит деление первого числа, на последующие, а результат выводит в файл. Если происходит деление на 0, то тогда дочерний и родительский процесс завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создает дочерний процесс.
- int pipe(int fd[2]); – создание канала; fd[0] — чтение, fd[1] — запись.
- int dup2(int oldfd, int newfd) - переназначение файлового дескриптора
- int execve(const char *path, char *const argv[], char *const envp[]) - запуск программы child
- ssize_t read(int fd, void *buf, size_t count) - чтение данных из файла/канала/терминала
- ssize_t write(int fd, const void *buf, size_t count) - запись данных в файл/канал/терминал.
- int open(const char *path, int flags, mode_t mode) - открытие/создание файла для записи результатов.
- int close(int fd) - закрытие файлового дескриптора
- pid_t waitpid(pid_t pid, int *status, int options) - ожидание завершения дочернего процесса.
- void _exit(int status) - немедленное завершение процесса

Также использованы функции стандартной библиотеки: strtof, strtok, strlen, fabsf.

Общий алгоритм решений:

В родительском процессе:

- читает строку чисел из stdin;
- записывает её в pipe1[1];
- читает односимвольный статус из pipe2[0]:
- '1' — строка обработана успешно;
- '2' — деление на ноль, нужно завершиться;
- '0' — потомок завершил работу по EOF.

В дочернем процессе:

- Открывает файл argv[1] через open с флагами O_WRONLY | O_CREAT | O_TRUNC.

- В цикле читает из stdin (связанного с pipe1) очередную строку.
- Разбивает её на токены через strtok с разделителями " \t\n".
- Первое число — делимое. Затем в цикле идут делители:
  если делитель равен нулю (fabsf(other) < EPS), записывает в pipe2 статус '2', закрывает файл и завершается через _exit(10);
  иначе выполняет накопительное деление result /= other.
- Полученный результат форматирует в строку собственной функцией ftostr (перевод float в десятичное представление без использования printf) и записывает в файл.
- Отправляет в pipe2 статус '1'.
- По завершении ввода (read вернул 0) отправляет статус '0' и завершается.

Код программы

src/parent.c:

```
#include <stdlib.h>

#include <unistd.h>

#include <sys/wait.h>

#include <string.h>


int main(void) {

    char filename[4096];

    ssize_t n;


    write(1, "Enter filename: ", 16);

    n = read(0, filename, sizeof(filename) - 1);

    if (n <= 0) _exit(1);

    if (filename[n-1] == '\n') n--;

    filename[n] = '\0';


    int pipe1[2], pipe2[2];

    if (pipe(pipe1) < 0 || pipe(pipe2) < 0) {

        write(2, "error: creating pipes\n", 22);

        _exit(2);

    }


    pid_t child = fork();

    if (child < 0) {
```

```
    write(2, "error: fork\n", 12);
    _exit(3);
}

if (child == 0) {
    close(pipe1[1]);
    close(pipe2[0]);

    if (dup2(pipe1[0], 0) < 0) { write(2, "error: dup2 stdin\n", 18); _exit(4); }
    if (dup2(pipe2[1], 1) < 0) { write(2, "error: dup2 stdout\n", 19); _exit(5); }

    close(pipe1[0]);
    close(pipe2[1]);

    char *args[] = {"./child", filename, NULL};
    extern char **environ;
    execve(args[0], args, environ);

    write(2, "execve failed\n", 14);
    _exit(6);
}

close(pipe1[0]);
close(pipe2[1]);

char buf[4096];
char status = 0;
const char *msg = "Input line <num num num>:\n";

write(1, msg, strlen(msg));
while ((n = read(0, buf, sizeof(buf) - 1)) > 0) {
    buf[n] = '\0';

    int ws;
    if (waitpid(child, &ws, WNOHANG) == child) break;
```

```
        if (write(pipe1[1], buf, n) < 0) break;

        if (read(pipe2[0], &status, 1) <= 0) { status = '2'; break; }
        if (status == '0' || status == '2') break;

        write(1, msg, strlen(msg));
    }

    close(pipe1[1]);
    close(pipe2[0]);

    waitpid(child, NULL, 0);

    if (status == '2') _exit(10);
    return 0;
}
```

src/child.c:

```
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <math.h>

#define EPS 1e-6f

int ftostr(float x, char * res) {
    int len = 0;
    int start = 0;
    if (fabsf(x) < EPS) {
        res[len++] = '0';
        res[len++] = '.';
        for (int k = 0; k < 6; k++) res[len++] = '0';
    }
```

```
else {
    if (x < 0) {
        res[len++] = '-';
        x = -x;
        start = 1;
    }

    int int_part = (int)x;
    float float_part = x - int_part + 5e-7f;
    if (int_part == 0) {
        res[len++] = '0';
    }
    else {
        while (int_part > 0) {
            res[len++] = '0' + int_part % 10;
            int_part /= 10;
        }

        int t;
        int count = len - start;
        int left, right;
        for(int j = 0; j < count / 2; j++) {
        left = start + j;
        right = len - 1 - j;
            t = res[left];
        res[left] = res[right];
        res[right] = t;
        }
    }
    res[len++] = '.';

    float_part *= 10;
    int_part = float_part / 1;
    if (int_part > 9) int_part = 9;
    float_part -= int_part;
```

```
        for (int k = 0; k < 6; k++) {
            res[len++] = int_part + '0';
            float_part *= 10;
            int_part = float_part / 1;
            if (int_part > 9) int_part = 9;
            float_part -= int_part;
        }
    }

    res[len] = '\0';
    return len;
}

int main(int argc, char *argv[]) {
    if (argc < 2) { write(2, "No filename\n", 12); _exit(1); }

    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { write(2, "Can't open file\n", 16); _exit(2); }

    ssize_t n;
    char *token;
    char buf[4096], str_result[64];
    float other, result;
    int len;

    while ((n = read(0, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = '\0';

        token = strtok(buf, " \t\n");
        if (token == NULL) { write(1, "1", 1); continue; }

        result = strtof(token, NULL);

        while ((token = strtok(NULL, " \t\n")) != NULL) {
            other = strtof(token, NULL);
```

```
            if (fabsf(other) < EPS) {
                write(1, "2", 1);
                close(fd);
                _exit(10);
            }
            result /= other;
        }

        len = ftostr(result, str_result);
        write(fd, str_result, len);
        write(fd, "\n", 1);

        write(1, "1", 1);
    }

    write(1, "0", 1);
    close(fd);
    return 0;
}
```

Протокол работы программы

```
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$ make
gcc -Wall -Wextra -Werror -Wfloat-equal -pedantic -o parent src/parent.c -lm
gcc -Wall -Wextra -Werror -Wfloat-equal -pedantic -o child src/child.c -lm
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$
```

Рисунок 1. Сборка программы

```
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$ ./parent
Enter filename: out.txt
Input line <num num num>:
50 25 2
Input line <num num num>:
1 10 10
Input line <num num num>:
0 5 5
Input line <num num num>:
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$ cat out.txt
1.000000
0.010000
0.000000
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$
```

Рисунок 2. Тест с нормальными данными

```
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$ ./parent
Enter filename: out.txt
Input line <num num num>:
5 0 5
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$ echo $?
10
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$ cat out.txt
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$
```

Рисунок 3. Тест с делением на 0

```
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$ ./parent
Enter filename: out.txt
Input line <num num num>:
-5 2
Input line <num num num>:
-100 25 4
Input line <num num num>:
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$ cat out.txt
-2.500000
-1.000000
user@DMITRY-PC:/mnt/c/Users/User/OS_2026$
```

Рисунок 4. Тест с отрицательными числами

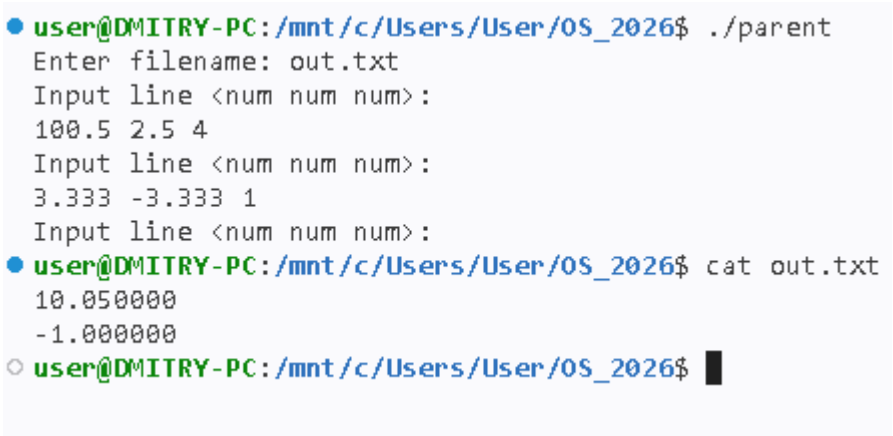


Рисунок 5. Тест с вещественными числами

Вывод

В ходе лабораторной работы были получены практические навыки управления процессами в ОС Linux и организации межпроцессного взаимодействия через каналы. Освоены системные вызовы fork, pipe, dup2, execve, read, write, open, close, waitpid, а также механизм перенаправления стандартных потоков ввода-вывода.